SOLUCIÓN – Seguimiento 2 (A) – Unidad 2 – Computación y estructuras discretas I

Funciones y relaciones

Se dan las siguientes nuevas definiciones sobre relaciones (usando la relación $E = \{(a,1),(b,2),(c,3)\}$ como referencia):

- dom: *dominio de definición de una relación.* Ej: $\operatorname{dom} E = \{a,b,c\}$.
- ran: *rango de una relación.* Ej: $\operatorname{ran} E = \{1,2,3\}$.
- R^T: *transpuesta de una relación*, invierte las tuplas. Ej: $E^T = \{(1,a),(2,b),(3,c)\}$.
- $R_2 \circ R_1$: *composición* de R_1 con R_2. Ej: $R = \{(1,1),(1,4),(2,3),(3,1),(3,4)\}$, $S = \{(1,0),(2,0),(3,1),(3,2),(4,1)\}$, $S \circ R = \{(1,0),(1,1),(2,1),(2,2),(3,0),(3,1)\}$.

1. Tome en cuenta las siguientes relaciones X, Y, Z:
   - $X = \{(1,2),(2,2),(3,1),(3,3)\}$
   - $Y = \{(a,b),(d,c),(e,f),(p,w)\}$
   - $Z = \{(2,g),(1,h),(c,7),(w,9)\}$

   Basándose en las definiciones anteriores, calcule: (a) $\operatorname{dom} X$ (b) $\operatorname{dom} Y^T$ (c) $\operatorname{dom} X \cap \operatorname{dom} Z$ (d) $\operatorname{dom} X \cup \operatorname{ran} X$ (e) $Z \circ X$ (f) $Z \circ X^T$

   (a) $\operatorname{dom} X = \{1,2,3\}$
   (b) $Y^T = \{(b,a),(c,d),(f,e),(w,p)\}$, por lo tanto $\operatorname{dom} Y^T = \{b,c,f,w\}$
   (c) $\operatorname{dom} X = \{1,2,3\}$ y $\operatorname{dom} Z = \{1,2,c,w\}$, por lo tanto $\operatorname{dom} X \cap \operatorname{dom} Z = \{1,2\}$
   (d) $\operatorname{dom} X = \{1,2,3\}$ y $\operatorname{ran} X = \{1,2,3\}$, por lo tanto $\operatorname{dom} X \cup \operatorname{ran} X = \{1,2,3\}$
   (e) Se buscan pares $(a,b) \in X$ y $(b,c) \in Z$:
   $(1,2) \in X,\ (2,g) \in Z \Rightarrow (1,g)$ $(2,2) \in X,\ (2,g) \in Z \Rightarrow (2,g)$ $(3,1) \in X,\ (1,h) \in Z \Rightarrow (3,h)$
   $(3,3) \in X$: no existe $(3,\cdot) \in Z$.
   $\therefore\ Z \circ X = \{(1,g),(2,g),(3,h)\}$
   (f) $X^T = \{(2,1),(2,2),(1,3),(3,3)\}$. Se buscan pares $(a,b) \in X^T$ y $(b,c) \in Z$:
   $(2,1) \in X^T,\ (1,h) \in Z \Rightarrow (2,h)$ $(2,2) \in X^T,\ (2,g) \in Z \Rightarrow (2,g)$ $(1,3)$ y $(3,3)$: no existe $(3,\cdot) \in Z$.
   $\therefore\ Z \circ X^T = \{(2,h),(2,g)\}$

2. Para cada función f con dominio $\mathbb{R}$ y codominio $\mathbb{R}$, determine si es invertible o no y argumente:
   (a) $f(x) = -3x + 4$
   (b) $f(x) = -3x^2 + 7$

   (a) **Sí es invertible.** Es una función lineal con pendiente $-3 \neq 0$, por lo que es biyectiva. *Inyectiva*: $f(x_1) = f(x_2) \Rightarrow -3x_1 + 4 = -3x_2 + 4 \Rightarrow x_1 = x_2$. ✓ *Sobreyectiva*: dado cualquier $y \in \mathbb{R}$, su preimagen es $x = \dfrac{4-y}{3} \in \mathbb{R}$. ✓
   (b) **No es invertible.** f no es inyectiva (contraejemplo): $f(1) = -3(1)^2 + 7 = 4$ y $f(-1) = -3(-1)^2 + 7 = 4$, pero $1 \neq -1$. Al no ser inyectiva, no puede ser biyectiva ni invertible.

Funciones

Nota: Dominio $\Leftrightarrow$ codominio se indica debajo de cada función.

3. Tome en cuenta las siguientes funciones:

$f(x) = \dfrac{1}{x^2}$	$g(x) = \dfrac{x}{x-8}$	$h(x) = \dfrac{x^2+4}{x}$	$m(x) = 3x + 5$	$n(x) = x^2 + 1$
$\mathbb{Z} \Leftrightarrow \mathbb{R}$	$\mathbb{R}-\{8\} \Leftrightarrow \mathbb{R}$	$\mathbb{R} \Leftrightarrow \mathbb{R}$	$\mathbb{Z} \Leftrightarrow \mathbb{Z}$	$\mathbb{R} \Leftrightarrow \mathbb{R}$

Rellene la tabla indicando **Sí** o **No**. En caso negativo, muestre el contraejemplo.

	¿Es inyectiva?	¿Es sobreyectiva?	¿Es biyectiva?
$f(x)$	**No.** $f(1) = f(-1) = 1$, pero $1 \neq -1$	**No.** $2 \notin \mathrm{ran}(f)$; el rango solo contiene positivos	**No**
$g(x)$	**Sí.** $g(x_1) = g(x_2) \Rightarrow x_1(x_2-8) = x_2(x_1-8) \Rightarrow x_1 = x_2$	**No.** $y{=}1 \notin \mathrm{ran}(g)$: $\frac{x}{x-8} = 1 \Rightarrow 0 = -8$, contradicción	**No**
$h(x)$	**No.** $h(1) = \frac{5}{1} = 5$ y $h(4) = \frac{20}{4} = 5$, pero $1 \neq 4$	**No.** $\mathrm{ran}(h) = (-\infty, -4] \cup [4, \infty)$; $1 \notin \mathrm{ran}(h)$	**No**
$m(x)$	**Sí.** $m(x_1) = m(x_2) \Rightarrow 3x_1 + 5 = 3x_2 + 5 \Rightarrow x_1 = x_2$	**No.** $6 \notin \mathrm{ran}(m)$: $3x + 5 = 6 \Rightarrow x = \frac{1}{3} \notin \mathbb{Z}$	**No**
$n(x)$	**No.** $n(1) = n(-1) = 2$, pero $1 \neq -1$	**No.** $\mathrm{ran}(n) = [1, \infty)$; $0 \notin \mathrm{ran}(n)$	**No**

4. Sean $f, g : \mathbb{R} \to \mathbb{R}$ definidas por $f(x) = 3x + 1$ y $g(x) = \dfrac{x-1}{3}$.
   (a) Calcule $(f \circ g)(x)$ y $(g \circ f)(x)$.
   (b) ¿Es $f \circ g = g \circ f$? Según lo visto en el curso sobre funciones compuestas e invertibles: ¿qué cree que está pasando y por qué dan dicho resultado particular?

(a) $(f \circ g)(x) = f\left(\dfrac{x-1}{3}\right) = 3 \cdot \dfrac{x-1}{3} + 1 = (x-1) + 1 = x. \quad (g \circ f)(x) = g(3x+1) = \dfrac{(3x+1)-1}{3} = \dfrac{3x}{3} = x.$

(b) **Sí**, $f \circ g = g \circ f$ (ambas igualan x). Esto *no* ocurre para cualquier par de funciones; es una propiedad exclusiva de los pares de funciones inversas. g es la **función inversa** de f: al componer en cualquier sentido se obtiene la identidad $\mathrm{id}(x) = x$, que es exactamente la condición que define a dos funciones como inversas entre sí.

---

5. **(2.50 pts) Problema práctico: Jugadores Activos — Solución**

Contexto. Un videojuego en línea tiene dos servidores. Cada servidor tiene su propia lista de jugadores registrados (por ID entero). El sistema central también mantiene una lista de jugadores **baneados** que no pueden participar. Se necesita obtener la lista de jugadores **activos**: todos los jugadores únicos de ambos servidores que **no** estén baneados, impresos en orden ascendente.

Referencia de operaciones de conjunto en Java:

Método	**Operación**	**Efecto sobre `setA`**
`setA.addAll(setB)`	$A \cup B$	Agrega a `setA` los elementos de `setB` que no estén ya presentes
`setA.retainAll(setB)`	$A \cap B$	Elimina de `setA` todo lo que **no** esté en `setB`
`setA.removeAll(setB)`	$A \setminus B$	Elimina de `setA` todo lo que **sí** esté en `setB`

Entrada:

Primera línea: `NA NB K` con $1 \leq \mathrm{NA} \leq 100\,000$, $1 \leq \mathrm{NB} \leq 100\,000$, $0 \leq K \leq 50\,000$

Segunda línea: `NA` enteros — IDs servidor A

Tercera línea: `NB` enteros — IDs servidor B

Cuarta línea: `K` enteros — IDs baneados ($1 \leq \mathrm{ID} \leq 10^9$, pueden repetirse)

Salida:

Primera línea: número de jugadores activos

Segunda línea: sus IDs en orden ascendente separados por espacio

Actividad. Implemente el método `jugadoresActivos` marcado con `TODO`. El resto del `Main.java` ya está dado y no debe modificarse.

Ejemplos:

Ejemplo 1

Entrada:

```
5 4 2
3 7 1 9 3
5 7 2 9
7 2
```

Salida:

```
4
1 3 5 9
```

Ejemplo 2 — Todos baneados

Entrada:

```
3 3 3
10 20 30
10 20 30
10 20 30
```

Salida:

```
0
```

Ejemplo 3 — Baneado inexistente

Entrada:

```
4 2 1
8 4 6 2
1 6
99
```

Salida:

```
5
1 2 4 6 8
```

Idea de la solución. Se construyen tres `TreeSet`: uno con los IDs del servidor A (`activos`), otro con los del servidor B (`setB`) y otro con los baneados (`setBan`). La unión $A \cup B$ se obtiene con `activos.addAll(setB)`, que incorpora los IDs de B ignorando los que ya estén presentes. Luego, la diferencia $(A \cup B) \setminus$ Baneados se obtiene con `activos.removeAll(setBan)`, que elimina en una sola llamada todos los IDs baneados que aparezcan en el conjunto. Como `activos` es un `TreeSet`, el resultado queda ordenado de forma ascendente automáticamente, sin ningún paso extra de ordenamiento.

```java
1  import java.util.*;
2
3  public class Main {
4
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7          int na = sc.nextInt(), nb = sc.nextInt(), k = sc.nextInt();
8
9          int[] servidorA = new int[na];
10         for (int i = 0; i < na; i++) servidorA[i] = sc.nextInt();
11
12         int[] servidorB = new int[nb];
13         for (int i = 0; i < nb; i++) servidorB[i] = sc.nextInt();
14
15         int[] baneados = new int[k];
16         for (int i = 0; i < k; i++) baneados[i] = sc.nextInt();
17
18         sc.close();
19
20         TreeSet<Integer> activos = jugadoresActivos(servidorA, servidorB, baneados);
21
22         System.out.println(activos.size());
23         StringBuilder sb = new StringBuilder();
24         for (int id : activos) sb.append(id).append(' ');
25         System.out.println(sb.toString().trim());
26     }
27
28     public static TreeSet<Integer> jugadoresActivos(int[] servidorA,
29                                                     int[] servidorB,
30                                                     int[] baneados) {
31         TreeSet<Integer> activos = new TreeSet<>();
32         TreeSet<Integer> setB    = new TreeSet<>();
33         TreeSet<Integer> setBan  = new TreeSet<>();
34
35         for (int x : servidorA) activos.add(x);
36         for (int x : servidorB) setB.add(x);
37         for (int x : baneados)  setBan.add(x);
38
39         activos.addAll(setB);
40         activos.removeAll(setBan);
41
42         return activos;
43     }
44 }
```

Líneas clave:

- **Línea 38**: `activos.addAll(setB)` — realiza la unión $A \cup B$; los duplicados entre servidores se descartan automáticamente.

- **Línea 39**: `activos.removeAll(setBan)` — realiza la diferencia $(A \cup B) \setminus \text{Baneados}$; los IDs baneados inexistentes en el conjunto se ignoran sin error.
- El uso de `TreeSet` garantiza orden ascendente sin necesidad de llamar a `Collections.sort`.